Clase 24 — La capa de red: forwarding y routing

Bloque: D (Red) · **Semana:** 8 · **Duración estimada:** 100 min **Correlativa:** Clase 23 (repaso de transporte + Simulacro 3) · **Sigue con:** Clase 25 (conmutación en la red: datagramas y circuitos virtuales) **Material de referencia:** Tanenbaum cap. 5 · RFC 791, RFC 1812, RFC 2328, RFC 4271, RFC 4632, RFC 9234, RFC 8986

🎯 Al terminar esta clase vas a poder

1. **Explicar** por qué el objetivo de la capa de red es la entrega **de sistema final a sistema final** y no de enlace a enlace, y por qué esa diferencia define el diseño de todo lo demás.
2. **Distinguir** las dos funciones clave —**forwarding** y **routing**— por ámbito, escala de tiempo, determinismo y lugar de implementación, y explicar **cómo interactúan** formando un circuito cerrado.
3. **Calcular** a qué puerto de salida sale un paquete aplicando *longest prefix match*, y explicar por qué la tabla de forwarding se indexa **por prefijo** y no por host.
4. **Distinguir** el plano de datos del plano de control y clasificar un diseño como **en banda** o **fuera de banda**, justificando qué pasa si falla el plano de datos.
5. **Explicar** qué es el *flooding*, cómo se controla (TTL, duplicados, *split horizon*, prune) y por qué **en unicast no hay flooding** pero en multicast sí.
6. **Justificar** por qué el router no implementa capas superiores y por qué el cortafuegos sí necesita mirar TCP y UDP.
7. **Argumentar** con números por qué "cantidad de hops" puede ser una mala métrica de enrutamiento.

> **Prerequisito del examen:** `2011-03 P1` (8 pts), `2009-02 P2` (10 pts), `2010-02 P2` (8 pts), `2008-07 P4` (10 pts), `2009-12 P3` (8 pts).

PARTE 1 — TEORÍA

1.1 El objetivo: de sistema final a sistema final

La capa 3 tiene **un solo cometido declarado**: **entregar cada paquete de un sistema final (host) emisor a un sistema final (host) receptor**, a través de una malla de redes intermedias que no son todas iguales ni todas conocidas.

	Capa 2 (enlace)	**Capa 3 (red)**
Entrega	nodo a nodo, en **un** enlace	sistema final a sistema final, a través de **muchos**
Problema	de alcance **único**	de alcance **global**
Pregunta que resuelve	¿a qué **MAC** mando esto para que llegue al router vecino?	¿por qué **interfaz** mando este paquete para que llegue a 8.8.8.8?
Punto final de la entrega	el router vecino	el **host** destino

Por qué esa diferencia define todo lo demás. (1) **El router es un modo de transporte, no el fin de la comunicación**: si la IP destino coincide con una de sus interfaces entrega localmente, si no, reenvía. (2) **El router no necesita entender el contenido**: sólo lee la dirección destino; si entendiera TCP tendría que demultiplexar por puertos y recalcular checksums, imposible a la velocidad del enlace. (3) **Aparece el concepto de ruta**: en capa 2 no hay rutas, hay "el vecino de al lado"; en capa 3 el camino tiene que **descubrirse** (routing) antes de poder **recorrerse** (forwarding). (4) **El servicio es *best-effort***: no garantiza entrega, ni orden, ni camino fijo por paquete, y esa falta de garantías es lo que permite que la red **escale**.

1.2 Las dos funciones clave

	Forwarding (reenvío) — plano de datos	**Routing (enrutamiento)** — plano de control
Qué es	mover un paquete del **puerto de entrada al de salida**	determinar la **ruta de origen a destino** e instalarla en las tablas
Ámbito	**local**: este router y este paquete	**global**: necesita información de los otros routers
Granularidad	**por paquete**	**por prefijo**
Tiempo	**nanosegundos–microsegundos**	**segundos**
Naturaleza	**determinista**: misma entrada + mismo destino = misma salida	**heurística**: caminos subóptimos y posibles ciclos
Dónde corre	**hardware**, en TCAM	**software**, en la CPU
¿Habla con pares?	**no**: un router aislado reenvía igual	**sí**, con al menos un protocolo de enrutamiento
Produce	la **FIB** (*Forwarding Information Base*)	la **RIB** (*Routing Information Base*), que se compila a la FIB

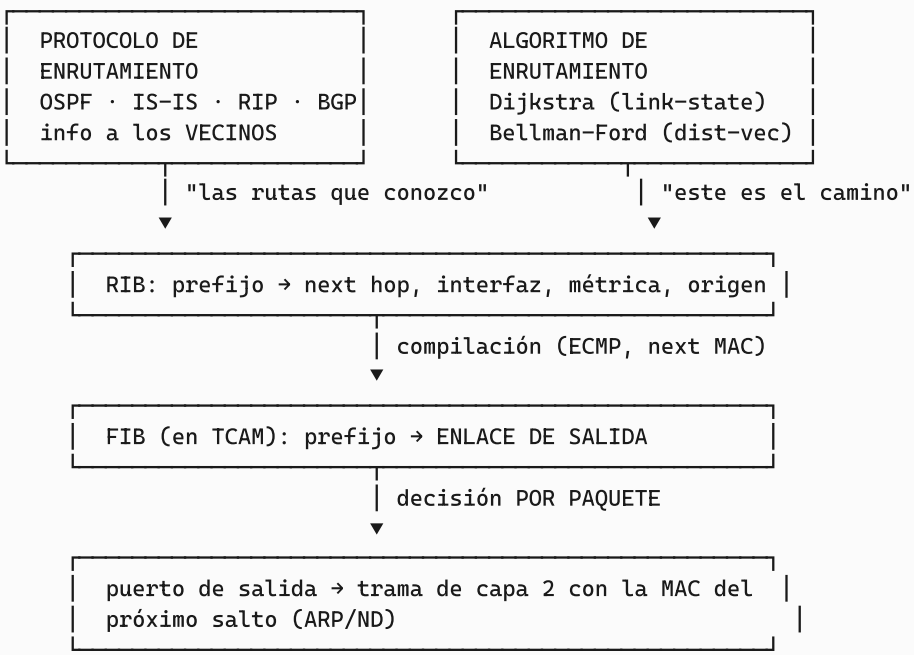
Por qué el forwarding no puede ser heurístico: si fuera aproximado, dos paquetes del mismo flujo podrían tomar caminos distintos y el **reordenamiento** sería el estado normal; y tiene que ser reproducible en silicio a

100 Gbit/s, o sea un número finito de comparaciones por paquete. La heurística vive **arriba**, en un plano lento donde gastar CPU no es un drama.

Por qué TCAM y no RAM común: la pregunta "¿qué entradas matchean esta dirección?" no es una búsqueda por índice sino una **comparación contra todas** (millones, en un router de núcleo). La **TCAM** (*Ternary Content-Addressable Memory*) guarda datos **y** máscara por entrada y compara **todas simultáneamente**, entregando el resultado en un ciclo. Como es carísima ⇒ **la FIB va en TCAM y la RIB (métricas, atributos, ECMP) en RAM.**

1.3 El circuito cerrado: protocolo → algoritmo → tabla de forwarding

Este es el esquema que pide el `2011-03 P1`:



Consecuencia práctica que se pregunta: si la FIB ya está poblada y el router se queda aislado del plano de control, **sigue reenviando con la tabla vieja**. Un router al que se le cae BGP **se congela, no se apaga**.

1.4 Longest prefix match: por qué la tabla es por prefijo

¿Por qué no por host? Una tabla por host tendría **2^{32} entradas**. Además **no haría falta**: si te dicen "la red `192.168.1.0/24` está directamente conectada", **una sola entrada** cubre 254 direcciones. Eso es la **agregación de prefijos** (CIDR, `RFC 4632`), y es lo que hizo posible que Internet escalara.

Si DOS entradas matchean, gana la más específica (la que tiene más bits de prefijo): *longest prefix match*. **¿Por qué la más específica y no la menos?** Porque la tabla se puebla con **excepciones**: "mi LAN entera está acá" y además "dentro de mi LAN hay una subred que **no** es mía". Si ganara la menos específica, ese paquete iría a la LAN local, que lo descartaría por ser de otra subred. **El longest match es lo que permite expresar excepciones.**

```
Tabla:  192.168.1.0/24 → eth2 (directa)
        192.168.0.0/16 → eth1 (next hop R1)
        0.0.0.0/0      → eth0 (next hop R2, default)

Busco 192.168.1.5:
  /24 → AND 255.255.255.0 = 192.168.1.0  ✓ matchea
  /16 → AND 255.255.0.0   = 192.168.0.0  ✓ matchea
  /0  → AND 0.0.0.0       = 0.0.0.0      ✓ matchea SIEMPRE
→ GANA /24 (24 > 16 > 0)  →  eth2.

Busco 192.168.77.9:  el /24 no matchea; matchean /16 y /0 → GANA /16 → eth1.
Busco 8.8.8.8:        sólo matchea /0 → eth0.
```

En TCAM **todas** las entradas matchean en paralelo y la más específica está marcada, por eso la tabla se ordena de `/0` a `/32` y la búsqueda es una **comparación**, no un algoritmo.

1.5 Plano de datos, plano de control: en banda y fuera de banda

El **plano de datos** mueve los paquetes según la tabla: por paquete, nanosegundos, determinista, en hardware. El **plano de control** calcula y distribuye qué tabla debe tener cada nodo: por prefijo, segundos, heurístico, en software. **Relación:** el control **escribe** la tabla que los datos **leen**; no se le consulta por paquete, sólo cuando la tabla cambia. Por eso el forwarding puede ser determinista y rapidísimo: **todo el trabajo de pensar ya está hecho de antemano**.

```
EN BANDA (in-band)                FUERA DE BANDA (out-of-band)
datos y control por el MISMO      datos por un camino, control
canal o camino                    por un canal DISTINTO

el control depende del dato:      el control tiene vida propia:
si se rompe el plano de datos,    si se rompe el plano de datos,
se rompe TAMBIÉN el control        el control sigue y puede
→ no hay por dónde diagnosticar   repararlo
```

- **En banda.** El **switch que aprende**: no tiene protocolo de control, aprende la MAC origen de las tramas que él mismo reenvía, así que su único plano es el de datos. Y el **enrutamiento en IP**: los LSA de OSPF y los UPDATE de BGP **son** paquetes IP de la misma red que rutean ⇒ plano de control **auto-referencial**. Si el plano de datos se rompe, no llegan las actualizaciones y **no hay forma de quejarse, porque el canal de queja es el que está roto**.
- **Fuera de banda.** El **LSP de MPLS**: LDP o RSVP-TE asignan una etiqueta a cada salto y los datos reenvían **sólo por etiqueta**; si LDP se cae, los paquetes con etiqueta siguen reenviéndose. La **red telefónica**: los circuitos de voz y la red de señalización son redes distintas, así que si cae la señalización no se puede establecer ninguna llamada **aunque todos los circuitos estén libres**. Un **enlace dedicado** o una **PVC de Frame Relay**: el circuito lo provisiona el operador y los datos no consultan nada durante toda su vida.

El desarrollo completo de esta pregunta, con la estructura de cada caso y la lista de ejemplos, está en **§3.3** (Examen 2008-07 P4).

1.6 Flooding

Flooding (inundación) es reenviar un paquete por **todos los puertos** (o todos menos el de entrada) en vez de consultar una tabla: no hay ACK, no hay estado, no hay tabla de "a quién le corresponde". **El problema:** en una red con ciclos —y toda red con caminos alternativos tiene— cada copia genera copias nuevas, así que sin control en *n* saltos circulan hasta **2^n** copias, el enlace se satura y la red se traba. **Por eso tiene que transformarse en controlado:**

Control	Cómo funciona	Contra qué
TTL / Hop Limit	Cada reenvío decrementa el TTL; al llegar a 0 se descarta y se avisa con ICMP *Time Exceeded* (tipo 11).	Acota el **radio** del flooding.
Detección de duplicados	Cada mensaje lleva un **identificador único** (origen + nº de secuencia); el nodo guarda los IDs vistos y **descarta** los repetidos.	Corta la **exponenciación** con ciclos.
Split horizon (horizonte dividido)	No le reenvíes a un vecino la ruta o el mensaje que **aprendiste de ese vecino**.	Evita la **realimentación** entre vecinos (conteo a infinito).
Exclusión del puerto de entrada	Nunca reenvíes por el puerto por donde entró la trama.	Evita devolver la trama al remitente.
Pruning (supresión)	Cuando se detecta que **no hay destinatarios** detrás de un enlace, se deja de inundar por ahí y se anuncia.	Deja de gastar ancho de banda en ramas muertas.

La combinación clásica es **flooding y prune en IP multicast**: cada router inunda, observa los **reportes IGMP** de sus subredes y, cuando detecta que una rama no tiene miembros, deja de inundar por ahí y anuncia el prune. En un switch es el **IGMP snooping** del `2010-12 P3`. El diagrama del flooding está en §3.2.

¿Y en unicast? En unicast no hay flooding. El reenvío de un datagrama unicast se hace **siempre por tabla** (longest prefix match, un puerto). Si la IP destino no matchea ninguna entrada, el paquete **se descarta** y se devuelve un ICMP *Destination Unreachable*; no se inunda. El flooding aparece en **link-state** (difundir las LSAs es el mecanismo principal) y en **multicast**.

1.7 El router no implementa capas superiores (y el cortafuegos sí)

Por qué el router se detiene en la capa 3. (1) **Rendimiento**: si entendiera TCP tendría que demultiplexar por el 5-tupla, mantener buffers por conexión y recalcular checksums, mientras que el reenvío de capa 3 es **una comparación de un entero** (a 100 Gbit/s entra un paquete cada 50 ns). (2) **No tiene el concepto de "conexión"**: TCP es extremo a extremo y un router intermedio es un tercero; guardar estado por flujo haría crecer su memoria con los flujos de toda Internet (es el error histórico de IntServ, Clase 25). (3) **Desacoplar las capas es el objetivo**: un router que no mira el contenido sirve para cualquier protocolo; si lo mirara dejaría de ser router y pasaría a ser cortafuegos. (4) **Es diseño explícito**: "el transporte es problema de los extremos".

Por qué el cortafuegos sí necesita TCP y UDP. Un cortafuegos **con estado** (*stateful*) mantiene una tabla de flujos con las columnas `{IP origen, puerto origen, IP destino, puerto destino, protocolo, estado}` — por ejemplo `10.0.0.5:51234 → 93.184.216:443 TCP ESTABLISHED` — y la regla es: si el paquete coincide con una entrada **existente** se permite (y la conexión "sale" sola); si no coincide, se consulta la ACL y sin permiso explícito se descarta. Esa entrada necesita IP de origen, IP de destino, **puertos** y **protocolo**: la IP es de capa 3, pero los puertos y el protocolo son de **capa 4**. Además tiene un *timeout* y distingue conexiones por los **flags**. Todo eso es información de TCP y UDP. Lo mismo pasa con el **NAT/PAT** (reescribe IP **y puerto**), con el

ALG (para que FTP pasivo funcione tiene que leer la cabecera FTP que va **dentro** del payload TCP: es el `2007-12 Problema 1`) y con los IDS/IPS de capa 7.

> **La regla general:** el router reenvía por dirección de capa 3; el cortafuegos decide por capa 3 **más** información de capas 4 y 7. Por eso un router no es un cortafuegos.

1.8 ⚠️ Actualización a 2026

Sigue igual, de memoria: forwarding por prefijo, en hardware, por longest prefix match · routing = un protocolo + un algoritmo que pueblan la tabla · la distinción plano de datos / plano de control · en banda y fuera de banda · el flooding como mecanismo de link-state y de multicast · la FIB en TCAM.

Cambió:

- **La seguridad del enrutamiento pasó de "confiar en el AS_PATH" a "validar el origen".** `RFC 6810` (RTR) y `RFC 6811` (BGP Prefix Origin Validation): el router valida que un prefijo esté autorizado por el RIR antes de aceptarlo. Es el ancla de confianza del enrutamiento, paralelo exacto de DNSSEC para el DNS. Y `RFC 9234` (may-2022) reemplaza el MD5 de sesión por **roles BGP** (Provider / Customer / Peer / Lateral / Internal) declarados en OPEN y UPDATE, para prevenir y detectar *route leaks*; `RFC 4271` sigue siendo la especificación vigente de BGP-4.
- **La separación control/datos pasó a ser una arquitectura.** SDN y **OpenFlow** (de la Open Networking Foundation, **no** del IETF: no tiene número de RFC) llevan el plano de control a un servidor que programa las FIBs ⇒ si te preguntan quién calcula la tabla y quién la aplica, la respuesta clásica (cada router calcula y aplica) tiene hoy alternativa: el cálculo puede estar centralizado.
- **Enrutamiento con estado sólo en el borde. Segment Routing** (`RFC 8402`, 2018; `RFC 8986`, SRv6) codifica las instrucciones de camino en **SID** que el nodo no tiene que interpretar: la FIB es "de prefijo largo a prefijo largo", **forwarding puro sin protocolo de control por salto**. SRv6 mete los SID **en la cabecera IPv6** y **elimina el MPLS por completo**.
- **El routing de datacenter dejó de ser BGP con link-state:** hoy es **EVPN-VXLAN** (`RFC 7432` + `RFC 7348`). **MPLS en datacenter fue reemplazado**; MPLS sigue **masivamente en carriers**. Y una FIB de núcleo tiene del orden de **millones de prefijos IPv4**: por eso la TCAM es cara, por eso existe la sumarización y por eso hay límites de tabla en BGP.
- **"Hops" dejó de ser la métrica por defecto en link-state.** OSPF define el costo de interfaz de forma configurable, con un valor por defecto derivado del **ancho de banda** (`RFC 2328`): `costo = ancho de banda de referencia / ancho de banda de la interfaz`. **Un enlace de 10 Gbps vale 1** con la referencia por defecto; uno de 10 Mbps vale muchísimo más ⇒ la métrica por defecto de OSPF **ya no cuenta hops, cuenta capacidad**.

Consecuencia para el examen: si el enunciado dice "la métrica es la cantidad de hops", respondé eso y sumá una línea: "*hoy la métrica por defecto de OSPF es un costo derivado del ancho de banda, no los hops; y en BGP la comparación es por longitud de AS_PATH*".

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — Longest prefix match con cinco entradas

Datos. El router S tiene esta tabla de forwarding:

#	Prefijo	Siguiente salto	Interfaz
1	`10.0.0.0/8`	directo	eth0
2	`172.16.0.0/12`	R2	eth1
3	`192.168.1.128/25`	R3	eth2
4	`192.168.1.0/24`	directo	eth3
5	`0.0.0.0/0`	R4	eth4

Se pide: a qué interfaz sale cada destino: `10.1.2.3`, `172.16.5.9`, `172.32.5.9`, `192.168.1.200`, `192.168.1.5`, `8.8.8.8`.

Resolución paso a paso.

Paso 1 — Las máscaras y la regla.

```
/8  → 11111111.00000000.00000000.00000000 = 255.0.0.0
/12 → 11111111.11110000.00000000.00000000 = 255.240.0.0
/25 → 11111111.11111111.11111111.10000000 = 255.255.255.128
/24 → 11111111.11111111.11111111.00000000 = 255.255.255.0
/0  → 00000000.00000000.00000000.00000000 = 0.0.0.0

Regla: AND bit a bit entre el destino y la máscara.
Si el resultado es IGUAL a los bits de red del prefijo → matchea.
De los que matchean, GANA el que tiene MÁS bits de prefijo.
```

Paso 2 — Los seis destinos. Cada celda es `destino AND máscara`; se marca si es igual a la red del prefijo.

Destino	AND /8	AND /12	AND /25	AND /24	AND /0	Gana	Interfaz
`10.1.2.3`	`10.0.0.0` **✓#1**	`10.0.0.0` ✗	`10.1.2.0` ✗	`10.1.2.0` ✗	`0.0.0.0` **✓#5**	**/8**	eth0
`172.16.5.9`	`172.0.0.0` ✗	`172.16.0.0` **✓#2**	`172.16.5.0` ✗	`172.16.5.0` ✗	`0.0.0.0` **✓#5**	**/12**	eth1
`172.32.5.9`	`172.0.0.0` ✗	`172.32.0.0` ✗	`172.32.5.0` ✗	`172.32.5.9` ✗	`0.0.0.0` **✓#5**	**/0**	eth4
`192.168.1.200`	`192.0.0.0` ✗	`192.168.0.0` ✗	`192.168.1.128` **✓#3**	`192.168.1.0` **✓#4**	`0.0.0.0` **✓#5**	**/25**	eth2
`192.168.1.5`	`192.0.0.0` ✗	`192.168.0.0` ✗	`192.168.1.0` ✗	`192.168.1.0` **✓#4**	`0.0.0.0` **✓#5**	**/24**	eth3
`8.8.8.8`	`8.0.0.0` ✗	`8.0.0.0` ✗	`8.8.8.0` ✗	`8.8.8.8` ✗	`0.0.0.0` **✓#5**	**/0**	eth4

Paso 3 — El caso que hay que saber hacer en binario.

```
172.32.5.9 = 10101100.00100000.00000101.00001001
/12        = 11111111.11110000.00000000.00000000
          AND= 10101100.00100000.00000000.00000000 = 172.32.0.0
La red de 172.16.0.0/12 = 10101100.00010000.… = 172.16.0.0
¿Coincide? NO → NO matchea.

192.168.1.200 AND 255.255.255.128 = 192.168.1.128 = red del /25  ✓ #3
192.168.1.200 AND 255.255.255.0   = 192.168.1.0   = red del /24  ✓ #4
GANA /25 (25 bits, el mayor número de bits) → eth2.
```

Interpretación del primer caso: el `/12` no es "los 12 primeros bits y los demás libres", es **doce 1s y veinte 0s, en orden**. El rango del `/12` es `172.16.0.0` a `172.31.255.255`. **Es el error de lectura de máscara más común del curso.**

Interpretación del segundo caso: `192.168.1.200` está en la red local de S (`192.168.1.0/24`), pero la tabla dice que **la mitad superior de esa LAN no es de S**: es una subred /25 que va para otro lado. Si S mandara el paquete a su LAN local, la máquina que lo recibiera lo descartaría por ser de otra subred. **El longest match salva el caso.**

⚠️ Errores típicos

- **Contar "cuántas entradas matchean" en vez de "quién tiene más bits".** En `10.1.2.3` ganó el /8 (8 > 0), no el /0.
- **Confundir el prefijo con la red.** `172.32.x.x` **no** está en `172.16.0.0/12`.
- **Olvidar que el default matchea siempre.** Sin entrada que matchee, el paquete **se descarta** (con ICMP *Destination Unreachable`), no se reenvía "por donde sea".
- **Tratar el `/n` como decimal.** `/25` son 25 unos en binario, no "el número 25".

2.2 Ejercicio 2 — ¿En banda o fuera de banda?

Enunciado. Clasificá cada diseño y decí qué pasa si el plano de datos se rompe. **(a)** Switch de capa 2 que **aprende la MAC origen** de cada trama que reenvía, sin ningún otro protocolo. **(b)** Router que corre **OSPF**: los Hello y los LSA son datagramas IP que viajan por la misma red que OSPF rutea. **(c)** Router que aprende rutas por **BGP**, protocolo propio sobre **TCP/179**; sus mensajes siguen siendo paquetes IP de la red que BGP rutea. **(d)** Switch de datacenter con un **puerto de gestión** en una red de administración distinta de la de producción.

Resolución.

	Clasificación	Por qué
(a)	**En banda**	No hay plano de control: el único plano es el de datos y la "tabla" se construye **mirando las tramas**. Si el switch se apaga se pierde entera y hay que reconstruirla con el tráfico; mientras tanto se inunda a los destinos desconocidos.
(b)	**En banda, auto-referencial**	Los LSA son paquetes IP que necesitan que las tablas **ya estén funcionando** para llegar a sus vecinos. Si el plano de datos se rompe no llegan, **el grafo queda desactualizado**, los Dijkstra dan caminos que no existen y **no hay forma de quejarse porque el canal de queja es el que está roto**.
(c)	**En banda en el medio, protocolo distinto en el plano lógico**	No está fuera de banda: sus mensajes necesitan IP funcionando para viajar. Pero BGP **es un protocolo de control distinto** del que lleva los datos de los usuarios y, al correr sobre TCP, **no depende de que IP sea fiable**: TCP le da ACKs, reintentos y orden. Y como es *path-vector* **no conoce la topología**: si un enlace se rompe deja de anunciar por ahí, pero **no calcula mal**.
(d)	**Fuera de banda**	La red de administración es un camino distinto con vida propia. Si se rompe todo el tráfico de producción seguís entrando por el puerto de gestión a mirar logs, recargar la configuración o sacar el equipo. Exactamente por eso los routers críticos tienen consola y gestión out-of-band.

```
EN BANDA       → el control usa el camino del dato. Si el dato muere, el
                 control muere con él. Ventaja: no hay que cablear nada
                 extra. Desventaja: no podés diagnosticar lo que no tenés
                 por dónde hablar.

FUERA DE BANDA → el control tiene camino propio: sobrevive al fallo del
                 dato y puede repararlo. Ventaja: reparás. Desventaja: hay
                 que construir y mantener el camino de control.
```

2.3 Ejercicio 3 — ¿Por qué "hops" es una mala métrica?

Enunciado. Dos caminos de A a Z: **camino 1** = 1 enlace directo de **10 Mbit/s** con 20 ms de propagación; **camino 2** = 5 enlaces de **1 Gbit/s** con 1 ms de propagación cada uno.

Resolución.

Paso 1 — La métrica por hops elige el camino 1 (1 < 5).

Paso 2 — Ancho de banda. El flujo va por el cuello de botella, o sea el enlace más estrecho de cada camino.

```
Camino 1: 10 Mbit/s
Camino 2: 1 Gbit/s = 1000 Mbit/s
→  1000 / 10 =  100 veces más ancho de banda en el camino de 5 hops.
```

Paso 3 — Retardo de transmisión de un paquete de 1500 bytes (trama Ethernet con carga IP máxima):

```
L = 1500 bytes × 8 bits/byte = 12 000 bits        d_trans = L / R

Camino 1: 12 000 / (10 × 10⁶) = 1,2 × 10⁻³ s = 1,2 ms   por enlace
Camino 2: 12 000 / (10 × 10⁹) = 12  × 10⁻⁶ s = 12 µs      por enlace
(1,2 ms = 1200 µs → el de 10 Mbit/s es 100× más lento)
```

Paso 4 — Propagación y retardo total de UN paquete, con `d_proc = 0` y sin cola:

```
Camino 1: 0 + 1,2 ms + 20 ms           = 21,2 ms
Camino 2: 0 + 5×0,012 ms + 5×1 ms     =  5,06 ms
→ el camino de 5 hops es 21,2 / 5,06 = 4,2 VECES mejor.
```

Interpretación, y es la frase para el examen: **"Hops" mide la cantidad de saltos, no la calidad del camino.** Un salto es un salto tanto si es de 10 Mbit/s como de 1 Gbit/s; la métrica **ignora por completo** las dos cosas que más afectan al usuario: el ancho de banda del cuello y el retardo acumulado. Y **peor**: como menos saltos suele significar menos procesamiento y menos encolado, la métrica por hops **tiende a elegir el camino corto**, que en la práctica es a menudo el de 10 Mbit/s y no el de 5 Gbit/s.

El único argumento a favor, y hay que decirlo: los hops **sí** correlacionan con el retardo de **procesamiento**, porque son nanosegundos (o microsegundos si el router está congestionado) por salto. Pero es de otro orden de magnitud que la diferencia de ancho de banda y que la de propagación: la balanza no está ni cerca.

Algoritmo	Métrica real
OSPF / IS-IS	**Costo de interfaz** configurable; por defecto derivado del **ancho de banda** (`RFC 2328`): `costo = ancho de banda de referencia / ancho de banda de la interfaz` (con referencia de 100 Gbit/s en equipos modernos, para que 1 Gbit/s no salga a costo 100).
RIP	**Cantidad de saltos**, máximo 15: sí cuenta hops, y es de lo más criticado por eso.
EIGRP	**Ancho de banda + retardo** (métrica compuesta).
BGP	Preferencia local → **longitud del AS_PATH** → origen → **MED**. No usa hops de router: los del interior del AS los aporta el IGP de abajo.

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2011-03, Pregunta 1 (8 puntos) — las dos funciones clave y su interacción

> **Fuente:** Examen `eirc1103` / `sirc1103` — 21 de febrero de 2011. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1103.pdf **Texto del examen:**
>
> > Describa claramente las dos funciones clave de la capa de red y explique cómo interactúan entre ellas.

Solución oficial (literal del PDF):

> **Reenvío (forwarding):** mover paquetes entre puertos de entrada y salida del router. **Enrutamiento (routing):** determinar la ruta de los paquetes desde origen a destino. Para ello existen los algoritmos de enrutamiento. **Interacción:** Cada router intercambia con sus pares, mediante al menos un protocolo de enrutamiento, información que se utiliza, en al menos un algoritmo de enrutamiento, para poblar sus tablas de forwarding. Dichas tablas, en su formato más básico, contiene, para un valor de dirección contenido en la cabecera de los paquetes que llegan al router, el enlace de salida. De esta forma, para cada paquete que arriba a un router por cualquiera de sus interfaces, el router examina la tabla de forwarding (en el caso de IP a partir de la dirección IP destino contenida en el encabezado IP) y a partir del "longest match" determina por qué interfaz debe ser enviado (además de determinar cuál es el "next hop", lo que determina en el caso más genérico, la dirección destino de capa de enlace que deberá contener la trama que lo lleve a su próximo salto). **Referencia:** `cap04_1_1xhoja_2009.pdf`, diapositivas 5 y 6.

Análisis de la solución oficial. Lo que está bien: la definición de forwarding es impecable y es exactamente el ámbito **local**; la interacción está en **tres eslabones** (**protocolo** → **algoritmo** → **tabla de forwarding**), con tres "al menos un" deliberados, porque son piezas **distintas** y puede haber varias de cada una; y menciona el **longest match** y el *next hop, que es el detalle fino: la tabla dice a qué interfaz y a qué dirección de router mandar, y de ese next hop se resuelve la MAC** de la trama.

Lo que **falta** y suma puntos: **la escala de tiempo y el lugar de ejecución** de cada una; **la distinción RIB / FIB**; **qué pasa si el routing falla** (si se cae el protocolo, **la tabla no se borra**: el router **se congela**); y que **el forwarding es determinista y el routing heurístico**, que es la razón de por qué la frontera entre los dos planos es hardware/SILICIO.

Mi respuesta completa, para tener a mano lo que me gustaría haber contestado:

> **Forwarding** es la función local que, para cada paquete que llega por una interfaz, lo mueve de la interfaz de entrada a la de salida. Es **por paquete**, en **nanosegundos**, y es **determinista**: dada una entrada de tabla y una dirección destino, la salida es siempre la misma. Se implementa en **hardware**, en una tabla

por prefijo (la FIB) que se consulta con **longest prefix match**. Si la IP destino coincide con una interfaz propia, se entrega localmente en vez de reenviarse.

Routing es la función que determina el camino de origen a destino y decide qué información se instala en las tablas de forwarding. Es **global** (necesita información de los otros routers), **por prefijo y no por paquete**, corre en **segundos** y es **heurística**: puede elegir caminos subóptimos y está sujeta a ciclos de convergencia. Se implementa en **software**, en el plano de control.

Interacción: cada router intercambia con sus pares, mediante al menos un **protocolo de enrutamiento** (OSPF e IS-IS son link-state e inundan; RIP es distance-vector; BGP es path-vector entre sistemas automáticos), información sobre la red. Esa información la procesa al menos un **algoritmo de enrutamiento** (Dijkstra para link-state, Bellman-Ford para distance-vector, más políticas y atributos) y el resultado se vuelca en la **tabla de enrutamiento (RIB)** local. De la RIB se compila la **tabla de forwarding (FIB)**, que en su forma más básica contiene, para un prefijo, el **enlace de salida**. Después, para cada paquete que llega por cualquier interfaz, el router toma la **dirección IP destino** del encabezado, busca el **longest match** en la FIB y reenvía por esa interfaz; con el **next hop** que esa entrada le da, resuelve la dirección de capa 2 del próximo salto para armar la trama.

Es un **circuito cerrado**: el plano de control alimenta al plano de datos, y por eso **la FIB puede seguir funcionando aunque el plano de control se caiga**: el router reenvía con la última tabla conocida hasta que una actualización nueva la reemplace.

Tope para 8 puntos: los dos axiomas (4) + el circuito cerrado con los tres eslabones y el longest match (4). Decir "el enrutamiento puebla las tablas de reenvío, que asocian destino con puerto de salida" ya te da la mitad. Sumar RIB/FIB y "el forwarding no se borra cuando cae el control" es lo que separa un 8 de un 6.

3.2 Examen 2009-02, Pregunta 2 (10 puntos) — forwarding, routing y flooding

Fuente: `eirc0902` / `sirc0902` — 2 de febrero de 2009. **Texto:**

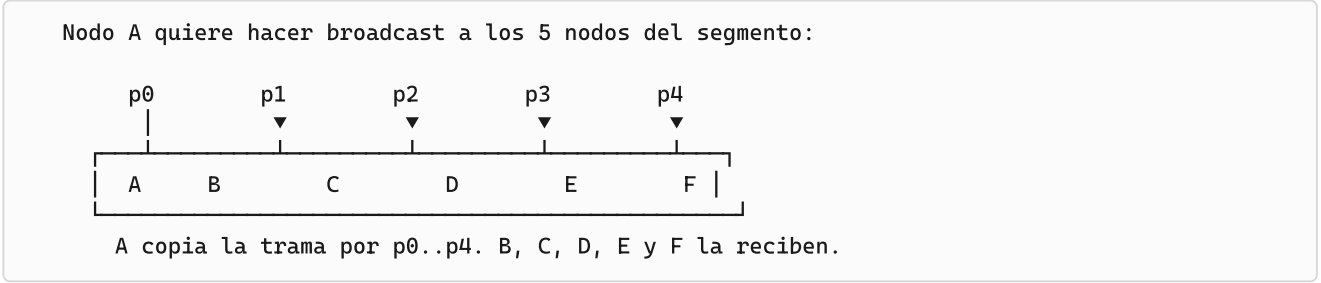
> a) En un contexto unicast, describa los procesos de forwarding y enrutamiento, y explique como se relacionan. b) En un contexto broadcast, describa un proceso de flooding, y explique como se puede transformar en flooding controlado.

Solución oficial (parte a, literal): "*El proceso de enrutamiento permite determinar el camino entre nodos de la red, típicamente mediante intercambio de información de topología y ejecución de un algoritmo en cada nodo [...] El proceso de forwarding consiste en copiar cada paquete entrante a un nodo al puerto de salida adecuado para que siga su camino en la red (o entregarlo a un proceso local si el destinatario es el propio nodo). La tabla de forwarding, que asocia destinos con puertos (o interfaces) del nodo/router, se construye en base a la información de enrutamiento.*" **La parte b) no tiene solución publicada en el PDF: lo que sigue es solución propia.**

a) La solución oficial trae los tres puntos que hay que decir: el routing "mediante intercambio de información de topología y ejecución de un algoritmo **en cada nodo**" (el "en cada nodo" importa: la información es **global**

pero el cálculo es **local**, no hay un centro); el forwarding "copiar cada paquete entrante al puerto de salida adecuado", más el paréntesis que vale puntos, **"o entregarlo a un proceso local si el destinatario es el propio nodo"**; y la relación, "la tabla de forwarding **se construye en base a** la información de enrutamiento". Sumá que en unicast el forwarding es **siempre por tabla** (nunca flooding), que un destino que no matchea ninguna entrada **se descarta**, y que el **longest prefix match** y el **next hop** son los dos pasos del desempate y del armado de la trama.

b) El emisor quiere que **todos** los demás nodos reciban su mensaje y **no sabe en qué puertos** están, así que no consulta ninguna tabla: **replica el paquete por todos los puertos** (o todos menos el de entrada), sin ACK, sin estado y sin tabla de "a quién le toca".



El problema, y por qué se transforma en controlado: en una red con ciclos cada copia genera copias nuevas, y sin control en *n* saltos circulan hasta $\mathbf{2^n}$ copias: el enlace se satura y la red se traba. Los cinco controles están en la tabla de §1.6 (**TTL/Hop Limit**, **detección de duplicados**, **split horizon**, **exclusión del puerto de entrada** y **pruning**). El caso real es **flooding y prune en IP multicast**: cada router inunda, observa los **reportes IGMP**, y cuando detecta que una rama no tiene miembros deja de inundar por ahí y anuncia el prune.

> **Tope para 10 puntos** (4 de a, 6 de b): en (a) los dos procesos, la construcción de la tabla y el "entregar a proceso local". En (b), **describir** el flooding sin ACK ni tabla, **nombrar el problema** y dar **al menos dos controles justificados**. Decir sólo "TTL" es un 6; dar el mecanismo + dos controles con su para qué es un 9-10.

3.3 Examen 2008-07, Pregunta 4 (10 puntos) — plano de datos vs. plano de control, en banda vs. fuera de banda

> **Fuente:** Examen `eirc0807` / `sirc0807` — 1 de agosto de 2008. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0807.pdf **Texto del examen:**
>
> > Explique la diferencia entre plano de datos (o forwarding) y plano de control. Mencione un ejemplo de protocolo de datos con control fuera de banda (con protocolo de control asociado) y otro con control dentro de banda (sin protocolo de control asociado).

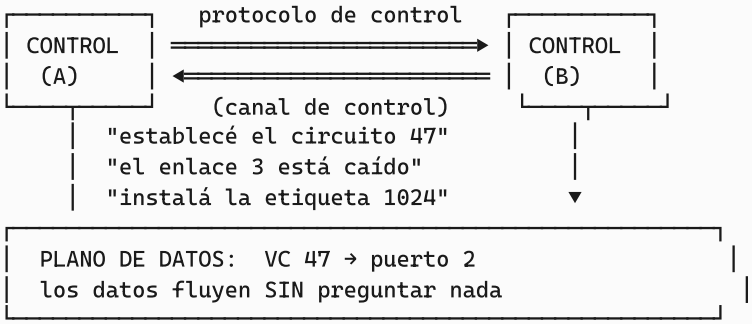
Nota: el PDF de solución **no incluye la parte teórica**. Todo lo que sigue es **solución propia**, sobre el esquema de §1.5.

La diferencia entre los dos planos

	Plano de datos	Plano de control
Qué hace	mueve los paquetes según la tabla	calcula y distribuye qué tabla debe tener cada nodo
Granularidad / tiempo	por paquete / nanosegundos	por prefijo / segundos
Ámbito / naturaleza	local / **determinista**	global / **heurístico**
Implementación	hardware (TCAM)	software (CPU)
Si falla	se corta el tráfico	**se congela**: la tabla vieja sigue sirviendo
Salida	FIB	RIB → compila a FIB

Relación: el plano de control **escribe** la tabla que el plano de datos **lee**; no se le consulta por paquete, sólo cuando cambia.

Control fuera de banda: estructura y ejemplos



Los tres rasgos: (1) **existe un protocolo de control asociado** con sus propios mensajes; (2) corre **antes o al margen** del flujo de datos; (3) los datos **no llevan la dirección destino**, llevan la **etiqueta** que el control les asignó.

- **Ejemplo A — MPLS y el LSP.** LDP o RSVP-TE establecen el camino y **asignan una etiqueta a cada salto**; después el plano de datos reenvía **sólo por etiqueta**, con un longest match de 20 bits, sin mirar la IP destino ni consultar ningún protocolo. Si LDP se cae, **los paquetes con etiqueta siguen reenviéndose**. Es el ejemplo más limpio porque el desacople es explícito: etiqueta (control) vs. IP (datos).
- **Ejemplo B — la red telefónica con señalización de canal común.** La voz viaja por circuitos dedicados conmutados; el control por una **red de señalización separada**. **Si se cae la red de señalización, no se puede establecer ninguna llamada aunque todos los circuitos de voz estén libres.**

Control en banda: estructura y ejemplos

No hay protocolo de control: **el plano de datos es el único plano** y todo lo que sabe sale de mirar el tráfico. En unicast hay una tabla construida por aprendizaje; en broadcast, flooding puro.

- **Ejemplo A — el puente con *self-learning*.** La tabla que dice qué MAC está en qué puerto se construye **mirando las tramas de datos que el puente reenvía** (lee la MAC origen y la asocia al puerto de entrada). Cuando un destino no está conocido, **inunda**; cuando una entrada envejece (típicamente 300 s en switches comerciales, valor **de fabricante**, no un estándar IEEE) la borra y vuelve a inundar hasta que alguien responda. Si el puente se apaga, se pierde todo y hay que reconstruirlo con el tráfico.

- **Ejemplo B — el reenvío unicast en IP.** El router reenvía por tabla, pero la tabla se construyó con protocolos que **viajan como paquetes IP por la misma red**. El plano de control es **auto-referencial**: para actualizarse necesita que el plano de datos de los demás funcione. Si se rompe un enlace, los vecinos se enteran porque **los mensajes dejan de llegar**: el acoplamiento es total.

La frase para cerrar los 10 puntos

> **En el plano de datos la decisión ya está tomada y es determinista; en el plano de control está por tomar y es heurística. "En banda" = el control viaja por el mismo canal que los datos, así que un fallo del dato se lleva el control y no hay forma de diagnosticar. "Fuera de banda" = el control tiene canal propio, así que sobrevive al fallo del dato y puede repararlo. Ejemplo en banda por excelencia: el puente que aprende de sus propias tramas. Ejemplo fuera de banda por excelencia: un LSP de MPLS, cuya etiqueta instala un protocolo de señalización antes de que circule ningún dato.**

3.4 Dos preguntas más que entran en esta clase

Son la misma materia con otro enunciado; las soluciones oficiales están en los PDF y son muy cortas, así que van resumidas. El desarrollo completo está en las clases correspondientes.

- `2010-02 P2` **(8 pts).** *"a) ¿Cuáles son las funciones clave de la Capa de Red? b) ¿Qué objetivo cumple cada una [...] y cómo interactúan? c) Describa brevemente las dos formas de prestar servicios de capa de red."* Oficial: *"Forwarding: mover paquetes entre puertos de entrada y salida del router. Enrutamiento: determinar la ruta de los paquetes desde origen a destino [...] El enrutamiento genera las tablas de forwarding locales."* **a)** forwarding y routing (§1.2). **b)** Oficial incompleto: falta el **objetivo de cada una** (forwarding resuelve *localmente y de inmediato* por qué interfaz sale cada paquete; routing determina *globalmente y con antelación* por dónde va el paquete e instala esa decisión en las tablas de los routers intermedios) y falta decir que **el routing no existe sin el forwarding**, porque el feedback de "esto no funciona" es lo que hace corregir a los algoritmos. **c)** Las dos formas de servicio de capa 3 son **conmutación de datagramas** (sin conexión, sin estado, decisión por paquete, caminos distintos por paquete, sin reserva) y **conmutación de circuitos virtuales** (con conexión, establecimiento previo, etiqueta en vez de dirección, estado en cada router, reserva posible) — es la Clase 25 entera.
- `2009-12 P3` **(8 pts).** *"a) ¿Cuál es el principal cometido de la capa de red? b) La longitud en 'hops' es la métrica utilizada habitualmente para definir el costo de un enlace. ¿Es la más conveniente?"* Oficial: *"Asegurar la entrega de información de extremo a extremo"* y *"la métrica de hops puede ser inconveniente, ya que señalará como 'mejor' el camino más corto, que no necesariamente será el que tenga mejores atributos de calidad (retardo extremo a extremo, pérdidas, throughput) y/o menos congestión"*. **a)** Correcta, con el matiz de que la capa de red hace **encaminar** *best-effort* y **no garantiza nada** (las garantías las da la capa de transporte). **b)** La oficial es buena; sumá que los hops **sí** correlacionan con el retardo de procesamiento (eso juega a favor de la métrica y hay que decirlo) pero es nanosegundos contra órdenes de magnitud en ancho de banda, y que hoy la métrica de link-state es un costo derivado del ancho de banda.

3.5 Nota de coordinación: Dijkstra es de la Clase 33

El **Problema 2 del** `sirc0902` es un ejercicio de **Dijkstra** paso a paso desde el nodo `u`, más el *shortest-path tree*. **No lo resolvemos acá**: es el tema de la **Clase 33 (enrutamiento link-state y OSPF)**. Lo que sí corresponde a esta clase es el **contenido** del flooding que Dijkstra necesita: cada nodo conoce la topología completa porque **se inundan las LSAs** (§1.6), y por eso link-state no escala sin jerarquías de áreas. Si te lo piden, el esqueleto es: inicializar `D(u)=0` y `D(v)=∞`; en cada iteración elegir el nodo no confirmado con `D` mínimo, **confirmarlo** y **relajar** sus aristas salientes con `D(v) = min(D(v), D(u) + c(u,v))`; repetir `n−1` veces.

PARTE 4 — CIERRE

📌 Resumen

1. La capa de red entrega **de sistema final a sistema final**. El router es un modo de transporte, no el destinatario.
2. **Forwarding** = local, por paquete, nanosegundos, determinista, en hardware (TCAM). **Routing** = global, por prefijo, segundos, heurístico, en software.
3. El **circuito cerrado**: protocolo de enrutamiento → algoritmo → RIB → FIB; la FIB se consulta **por paquete** con **longest prefix match**.
4. La tabla es **por prefijo** porque hace falta para agregar, escalar y meter en TCAM. Si matchean varias entradas, **gana la más específica**.
5. **En banda**: el control usa el camino del dato, y un fallo del dato se lleva el control. **Fuera de banda**: el control tiene camino propio y puede reparar al dato.
6. **Flooding** = reenviar por todos los puertos; se controla con **TTL**, **duplicados**, **split horizon**, exclusión del puerto de entrada y **pruning**. En **unicast no hay flooding**; en multicast sí.
7. El **router no implementa capas superiores**; el **cortafuegos sí** necesita puertos y flags porque decide por flujo.
8. **"Hops" es mala métrica**: un salto no pondera ancho de banda ni retardo, así que el camino de menos saltos puede ser el de 10 Mbit/s contra otro de 5 Gbit/s.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. ¿Por qué el objetivo de la capa de red es "de sistema final a sistema final" y no "de enlace a enlace"?

<details>
<summary>Respuesta</summary>
</details>

2. En una red con `192.168.1.0/24 → eth2`, `192.168.0.0/16 → eth1` y `0.0.0.0/0 → eth0`, ¿por qué puerto sale `192.168.1.5`? ¿Y si se agrega `192.168.1.128/25 → eth3`?

<details>
<summary>Respuesta</summary>
</details>

3. ¿Qué le pasa a un router al que se le cae el protocolo de enrutamiento? ¿Por qué sigue reenviando?

<details>
<summary>Respuesta</summary>
</details>

4. ¿"En banda" y "fuera de banda"? ¿Cuál permite reparar una red caída y por qué?

<details>
<summary>Respuesta</summary>
</details>

5. ¿En unicast hay flooding? ¿Y qué hace el router con un paquete unicast que no matchea ninguna entrada?

<details>
<summary>Respuesta</summary>
</details>

6. ¿Por qué el router no implementa las capas superiores y el cortafuegos sí?

▶ Respuesta

7. ¿Por qué los protocolos de enrutamiento de IP son "en banda"?

▶ Respuesta

8. Dos caminos al mismo destino: uno de 2 saltos a 10 Mbit/s y otro de 4 saltos a 1 Gbit/s. ¿Cuál elige un algoritmo que cuenta hops? ¿Es el correcto?

▶ Respuesta

🔗 Referencias

- **Examen** `eirc1103` **/** `sirc1103` — P1 (8 pts): las dos funciones clave de la capa de red y su interacción. La pregunta más repetida del banco sobre este tema.
- **Examen** `eirc0902` **/** `sirc0902` — P2 (10 pts): forwarding vs. routing en unicast; flooding en broadcast y controlado. Su Problema 2 es de Dijkstra y va en la Clase 33.
- **Examen** `eirc1002` **/** `sirc1002` — P2 (8 pts): funciones clave, objetivos y las dos formas de servicio de capa 3. · **Examen** `eirc0912` **/** `sirc0912` — P3 (8 pts): cometido de la capa de red; ¿es "hops" buena métrica?
- **Examen** `eirc0807` **/** `sirc0807` — P4 (10 pts): plano de datos vs. plano de control, en banda vs. fuera de banda. · **Examen** `sirc0712` — Problema 1 (30 pts): leer los puertos TCP/UDP dentro del paquete IP; capa 7 en un dispositivo intermedio.
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 5 "La capa de red".
- **RFC 791** — Internet Protocol (1981). · **RFC 1812** — Requirements for IP Version 4 Routers (1995). · **RFC 4632** — CIDR: An Address Assignment Plan for IPv4 (2006).
- **RFC 2328** — OSPFv2 (1998): costo de interfaz por ancho de banda y flooding de LSAs.
- **RFC 4271** — BGP-4 (2006). · **RFC 9234** — BGP Roles (2022). · **RFC 6810 / RFC 6811** — RPKI/ROA.
- **RFC 8402** — Segment Routing Architecture (2018). · **RFC 8986** — SRv6 Network Programming (2022). · **RFC 7432 / RFC 7348** — EVPN / VXLAN.
- **Clase siguiente:** 25 — Conmutación en la red: datagramas y circuitos virtuales